# Software architecture and image compression

## An implementation guide for the four-mode LoRa emergency communication prototype

Prepared 18 October 2026 from the local source. Nine original vector diagrams explain the system, software modules, compression pipeline, message lifecycle, four transfer modes and persistent acknowledgement path.

The diagrams represent the current implementation, not a proposed redesign. Radio settings are 915 MHz, 125 kHz bandwidth, SF9, coding rate 4/7, configured output power 22 dBm and an eight-symbol preamble. The current adaptive revision is adaptive-goodput-1.

This is a supporting explanation pack, not the complete ten-page FYP paper. For the paper, select the system architecture, compression flow and adaptive policy figures; use the mode comparison and storage details where they support the argument. Captions and editable SVG files are supplied for reuse.

Completed experiments and performance gains are not invented here. The two previously reported image times used unequal payloads and are unsuitable for attributing a speedup to the protocol. No firmware changes, board uploads or radio tests were performed for this diagram task.

| Reading path | What it explains |
|---|---|
| Sections 1-2 | Deployment, modules and runtime scheduling |
| Sections 3-4 | Exact compression decisions, pixel math and phone memory |
| Section 5 | Local save versus destination delivery |
| Sections 6-9 | Baseline, tuned, fixed and adaptive transfer |
| Sections 10-11 | Persistent ACKs, timing boundaries and research interpretation |

# 1. System architecture

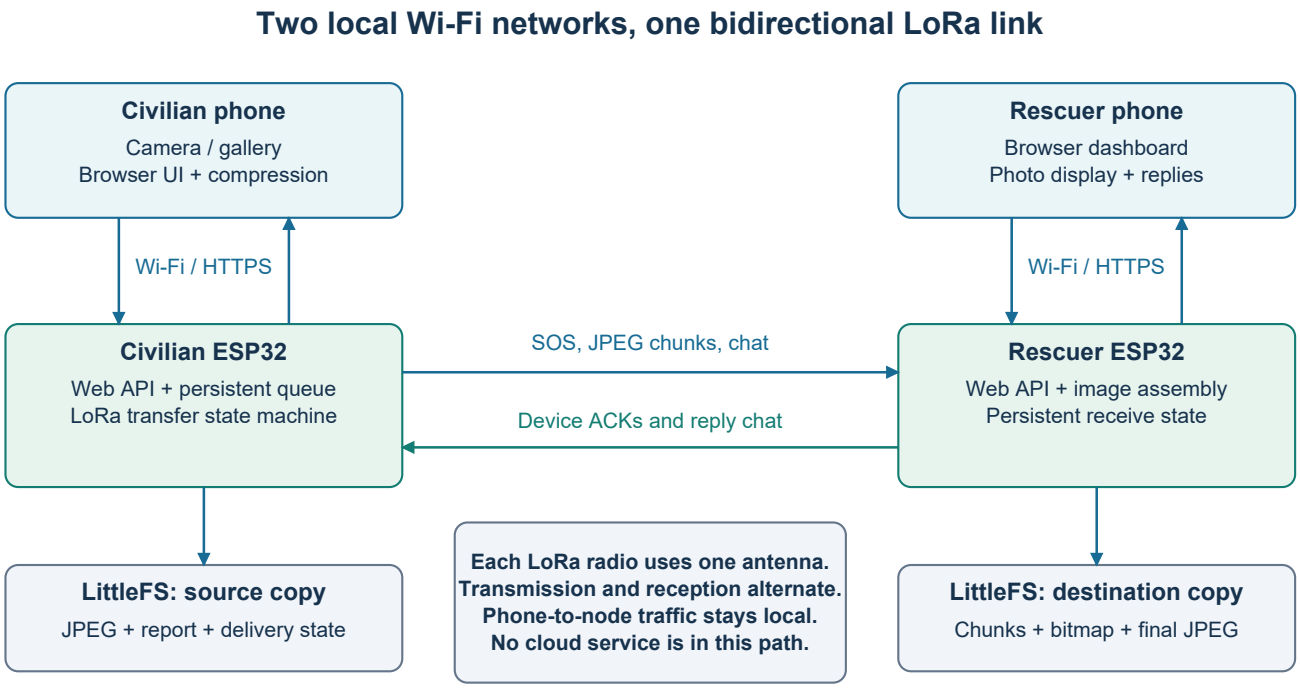



*Figure 1. Deployment and data paths of the current two-node system. The radio link carries both application data and destination acknowledgements.*

The civilian phone connects to the civilian node and the rescuer phone connects to the rescuer node. Both nodes host their own local web interface. The phones do not communicate directly across the LoRa link: the ESP32 firmware stores application records, creates radio frames and interprets replies.

A single firmware codebase is compiled with a civilian or rescuer role. The civilian can originate SOS reports and attach one image per report. Both roles can originate chat within an established report conversation. The rescuer receives and reconstructs photos, replies to chat and can mark a report resolved.

The current server code uses HTTPS on port 443, an HTTP-to-HTTPS redirect on port 80, and a DNS portal service. Wi-Fi association is open; the rescuer dashboard has a separate application login. These diagrams follow the active server code even where older interface wording mentions HTTP.

The radio is half duplex: it alternates between transmit and receive using the same LoRa antenna. A radio transmission finishing locally only means the frame left the sender. Destination acknowledgement is a later event, after the receiver processes and saves the data.

Source trace: testing.ino:17; WebServerManager.h:270; LoRaMeshManager.h:304

# 2. Software modules and runtime

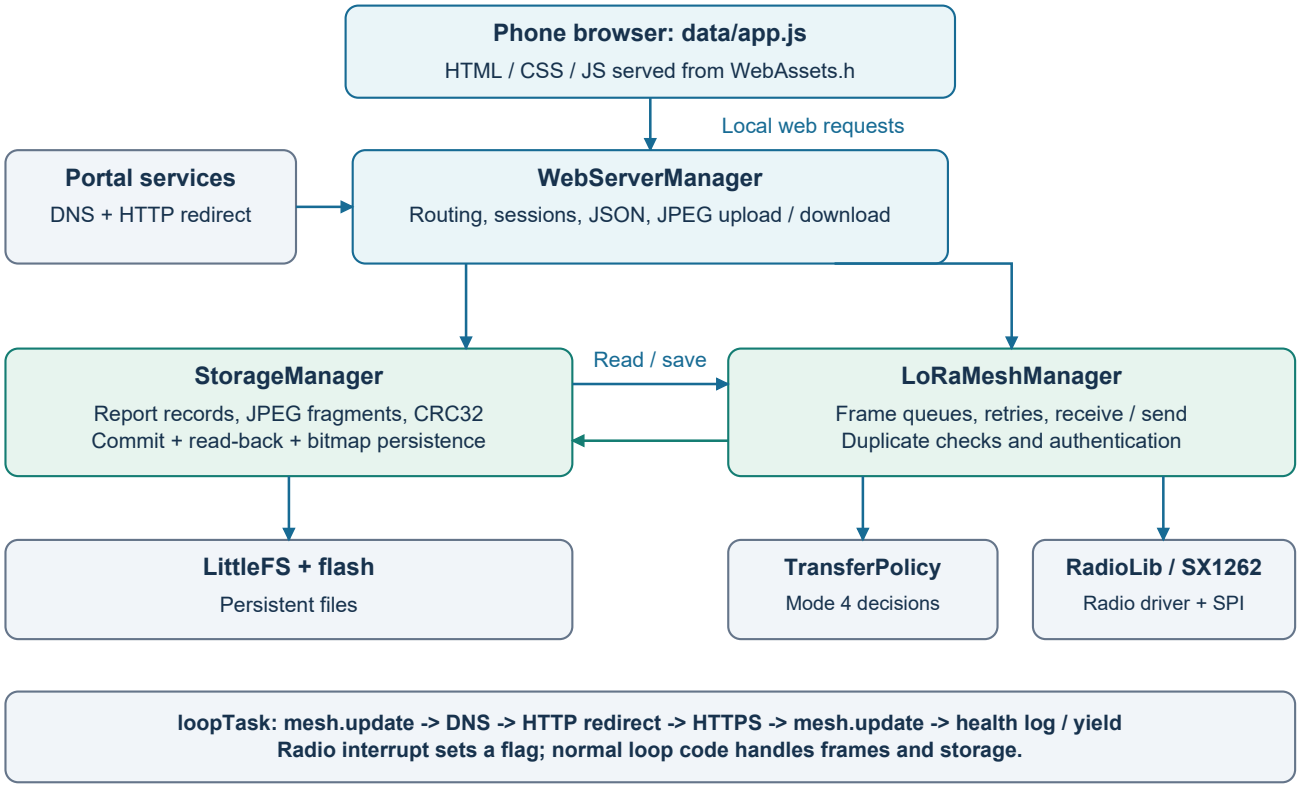



*Figure 2. Software responsibilities and cooperative execution. Boxes are logical modules, not separate parallel threads.*

The editable web source is in data/index.html, data/app.js and data/style.css. An embedding utility generates WebAssets.h, placing those resources in firmware flash. The ESP32 serves the JavaScript, but the connected browser executes it. WebServerManager validates requests, checks session access and exposes reports, image uploads and state queries.

StorageManager owns persistent records and image files. LoRaMeshManager owns a 16-slot transient frame queue, retry state, duplicate suppression, destination routing and radio operation. PacketDefs specifies binary fields and frame validation; TransferConfig selects a mode; TransferPolicy supplies airtime estimates and adaptive decisions. RadioLib controls the SX1262 through SPI.

Application handlers execute sequentially on loopTask. web.update calls mesh.update before and after DNS/web service. Long image reads and writes also call mesh.update at buffer boundaries. The radio interrupt sets a flag; the normal loop reads the radio and processes frames. The update loop additionally polls radio interrupt status to recover missed notification edges.

This shared execution model explains why a slow web operation can delay radio service. Cooperative calls reduce that delay but do not create an independent radio task or a hard latency guarantee. The loop stack is configured to 16 KiB; health logs expose minimum stack headroom, heap availability and the longest observed web/radio service gaps.

Source trace: data/app.js:1; WebServerManager.h:22,316; LoRaMeshManager.h:18,552; Config.h:37

# 3. Image-compression architecture

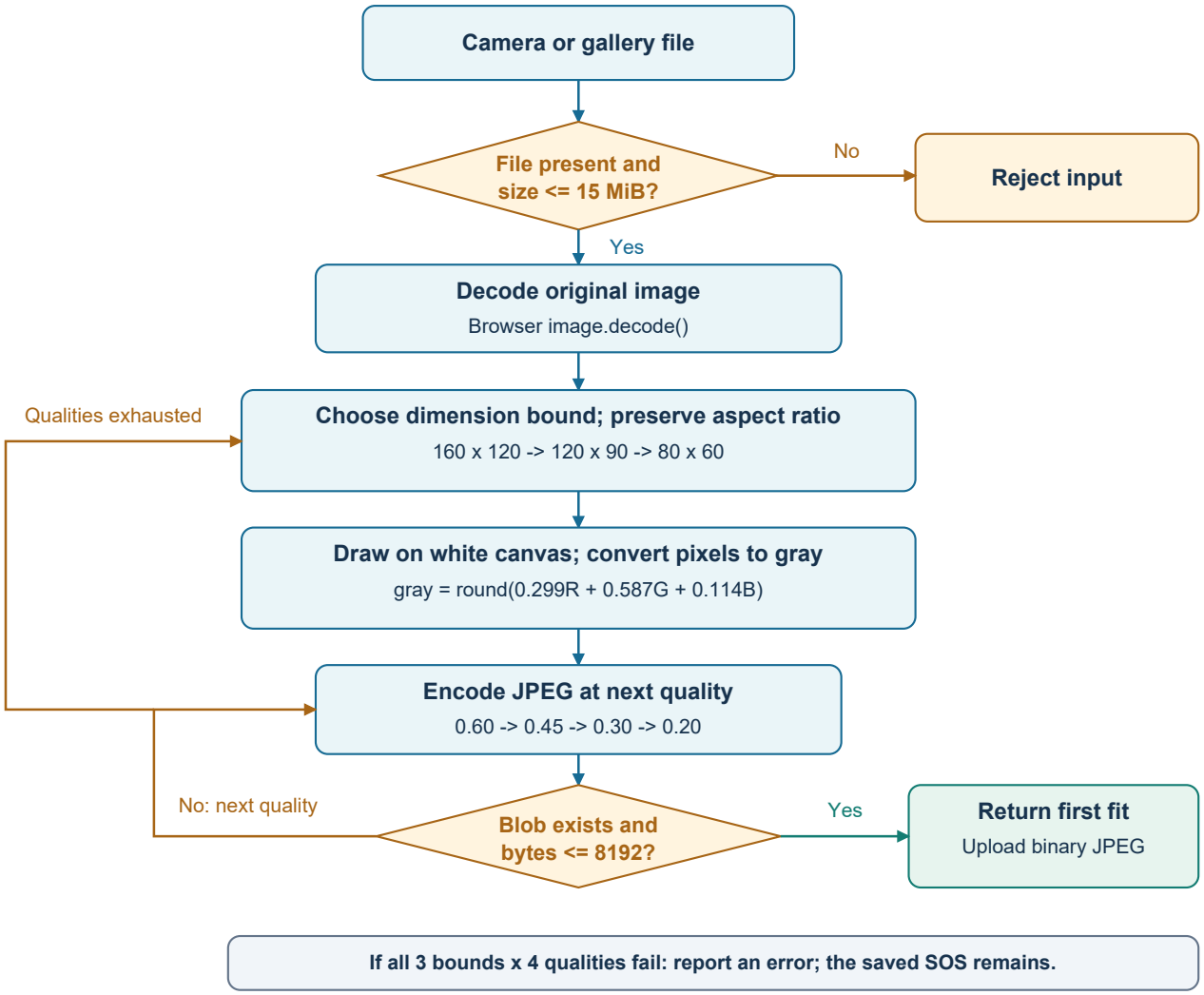



*Figure 3. Exact search order in compressPhoto(). The outer loop reduces dimensions; the inner loop tries JPEG quality hints. Every dimension candidate is drawn from the decoded original.*

Compression happens inside JavaScript running in the phone browser. The file picker returns a camera or gallery File. The function rejects missing input and files larger than 15 x 1024 x 1024 bytes, then creates an object URL and awaits image.decode(). It revokes the URL after decode; the decoded image remains available for drawing.

The outer loop tries bounding boxes of 160 x 120, 120 x 90 and 80 x 60 pixels. At each size it draws the image on a white canvas, converts the pixels to gray, and tries JPEG quality hints of 0.60, 0.45, 0.30 and 0.20. The first non-null Blob of at most 8192 bytes is returned. If every candidate fails, the photo operation reports an error; an SOS already saved locally remains stored.

The loop therefore performs at most twelve JPEG encoding attempts. It favours the larger bounding box before trying the next smaller one. It stops at the first acceptable file rather than comparing every candidate for perceptual quality. The ESP32 receives the final binary JPEG and does not perform this resizing or JPEG encoding.

Source trace: data/app.js:60-83; WebAssets.h:78-101

# 4. Compression: pixels, bytes and memory

**Resize rule.** For original dimensions W x H and a candidate bound Bw x Bh, the scale is s = min(Bw/W, Bh/H, 1). Output dimensions are max(1, round(Ws)) x max(1, round(Hs)). The aspect ratio is preserved and smaller images are not enlarged. A 4000 x 3000 photo becomes 160 x 120 at the first bound. A 3000 x 4000 portrait becomes 90 x 120, not 160 x 120.

**Pixel conversion.** After drawing, the code calculates gray = round(0.299R + 0.587G + 0.114B) for each canvas pixel and writes that value into R, G and B. The white background handles transparent input. The canvas still uses RGBA pixel data; equal colour channels do not turn it into a one-byte-per-pixel buffer.

**JPEG encoding.** canvas.toBlob delegates encoding to the browser. The quality argument is an encoder hint: 0.60 does not mean that the file will be 60% of the original size or have a measured 60% visual quality. The application checks the actual Blob size after encoding. It does not configure quantisation tables or chroma-subsampling settings, and equal R/G/B input does not itself prove a single-component grayscale JPEG file.

**Why a small output can still need substantial memory.** The full original is decoded before resizing. A 4000 x 3000 image represented as four bytes per pixel occupies approximately 48,000,000 bytes, or 45.8 MiB, for one pixel buffer alone. Browser, camera, encoder and duplicate buffers add overhead. The 15 MiB input check limits compressed file bytes, not decoded pixel count. A low-memory error after camera capture can therefore arise on the phone before the board sees the final JPEG; this code trace does not diagnose every such browser failure.

**Where the savings occur.** Fewer pixels and lossy JPEG encoding reduce the bytes placed on the LoRa link. The receiver reconstructs the same small JPEG. It does not restore discarded resolution or colour detail. This is appropriate to evaluate as a constrained situational photo, not as preservation of the original full-resolution image.

| Representation | Illustrative size | Location |
|---|---|---|
| Original 4000 x 3000 pixels | Compressed file size varies | Phone file / image decoder |
| One decoded RGBA buffer | 48,000,000 bytes (45.8 MiB) | Phone browser memory |
| 160 x 120 RGBA canvas | 76,800 bytes (75 KiB) | Phone browser memory |
| Accepted JPEG Blob | At most 8192 bytes (8 KiB) | Uploaded to civilian ESP32 |

Source trace: data/app.js:60-77; WebServerManager.h:170-199

# 5. SOS, image upload and delivery

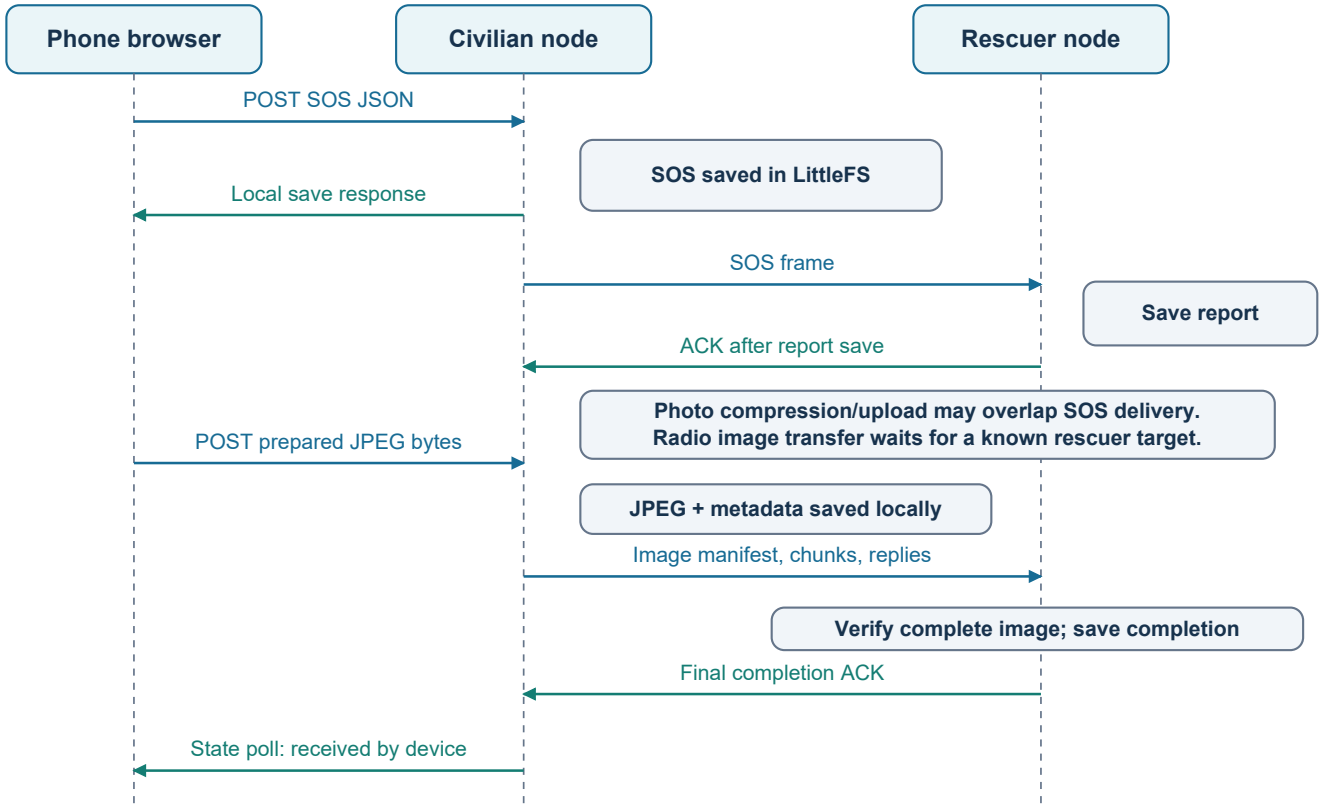



*Figure 4. The local web response, rescuer SOS acknowledgement and final image acknowledgement are separate milestones. Compression and upload can overlap SOS delivery.*

Pressing Save and send SOS first sends JSON to the civilian web API. After the report is saved, the browser receives a local response. If a photo was selected, the browser then compresses and uploads it. A local upload response means the source JPEG and metadata were saved on the civilian node; it does not establish radio delivery.

The upload handler accepts image/jpeg, checks the 8192-byte limit and available storage, and reads the request in 512-byte buffers. It computes CRC32 while writing a temporary .part file, checks the stored file, renames it to .jpg and saves the image record. The report can have one immutable image; retrying an already-existing attachment returns that image record.

The image scheduler waits until the associated SOS has been acknowledged and a rescuer target is known. It then exchanges the image manifest and chunks using the selected transfer mode. The source record can already say that its local file is complete while delivery is still pending: local file completeness and remote delivery are different fields.

After the rescuer verifies and saves the final image, its completion acknowledgement lets the sender mark delivery received. The browser periodically requests state; normal visible-page polling is scheduled every five seconds. A UI stopwatch can therefore include polling delay and web latency in addition to radio delivery time. Serial START/DELIVERED events provide a more precise protocol measurement.

Source trace: data/app.js:216-224,260; WebServerManager.h:170-199; LoRaMeshManager.h:416-421

# 6. Mode 1: baseline stop-and-wait

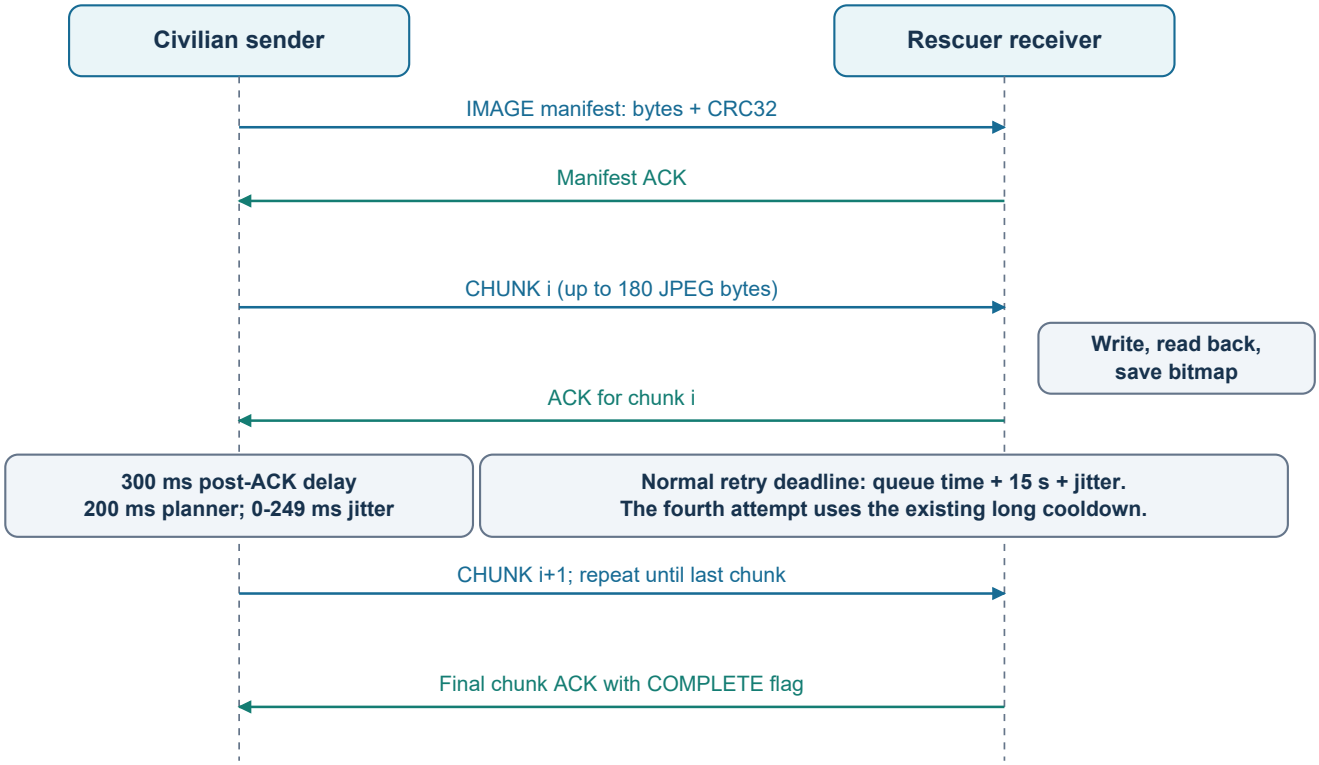



*Figure 5. Mode 1 sends one image chunk and waits for its acknowledgement before advancing. Manifest and final-completion exchanges are part of the image transfer.*

An IMAGE manifest identifies the associated report, expected JPEG byte count and CRC32. After the receiver accepts it, the sender transmits the lowest unacknowledged chunk. A normal chunk acknowledgement advances the stored bitmap. The final chunk acknowledgement carries COMPLETE only after image verification and completion-state persistence.

The baseline retains a 200 ms planning interval, a 300 ms post-image-ACK delay and transmit jitter in the range 0-249 ms. Its ordinary retry deadline is set from queueing, using 15 s plus 0-1999 ms jitter. The fourth queued attempt enters the existing approximately 120 s cooldown. An ACK can still complete the pending operation during that cooldown.

Stop-and-wait is simple because only one chunk is awaiting acknowledgement. Its cost is repeated turnaround: data airtime, receiver processing, ACK airtime, scheduling delay and any retry wait are paid for each chunk. A lost acknowledgement can produce a repeated chunk even when the receiver already stored it.

For an 8192-byte JPEG, 46 chunks are needed: 45 full chunks of 180 bytes and a final 92-byte chunk. This is an arithmetic example, not the byte count of the user's earlier photographs. One-at-a-time acknowledgement is unchanged by the image's visual contents.

Source trace: LoRaMeshManager.h:154-185,405-457; Config.h:44-47

# 7. Mode 2: tuned stop-and-wait

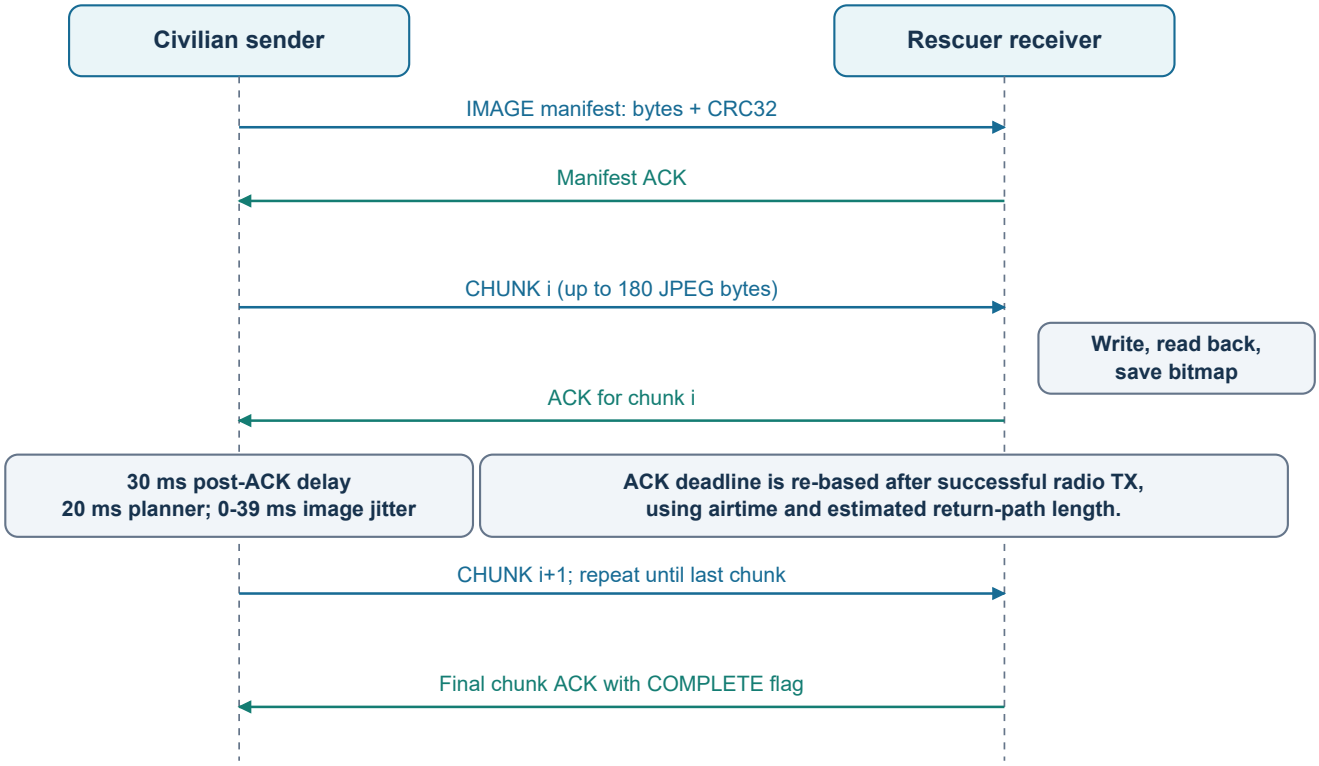



*Figure 6. Mode 2 keeps individual chunk acknowledgements and shortens scheduling delays. The sequence is unchanged; timing and receive handling differ.*

Mode 2 preserves the manifest, individual chunk ACK and final verification path. The planner runs on a 20 ms interval; image progress uses a 30 ms post-ACK delay and image-related enqueue jitter of 0-39 ms. These settings reduce avoidable idle time without changing the JPEG or radio modulation.

After a successful image-frame transmission, the acknowledgement deadline is re-based on calculated request/ACK airtime and the estimated route length. The return-path estimate comes from ACK TTL with a configured lower bound. A four-attempt cooldown remains. SOS and chat do not automatically inherit every image-specific timing rule.

Modes 2-4 also share receive-in-progress protection: before starting channel assessment, the scheduler checks whether a valid incoming packet is still being received and allows it time to finish. Channel assessment can otherwise switch the radio out of receive mode. A busy channel defers the queued frame.

This mode tests whether delay reduction alone helps. It still pays an acknowledgement exchange per chunk, so it cannot remove the per-chunk round-trip structure. Faster timing can help on a stable link, while real delays from web service, storage, radio contention and retransmission still remain.

Source trace: TransferPolicy.h:20-27; LoRaMeshManager.h:142-144,459-474,614-629

# 8. Mode 3: fixed groups and bitmap ACKs

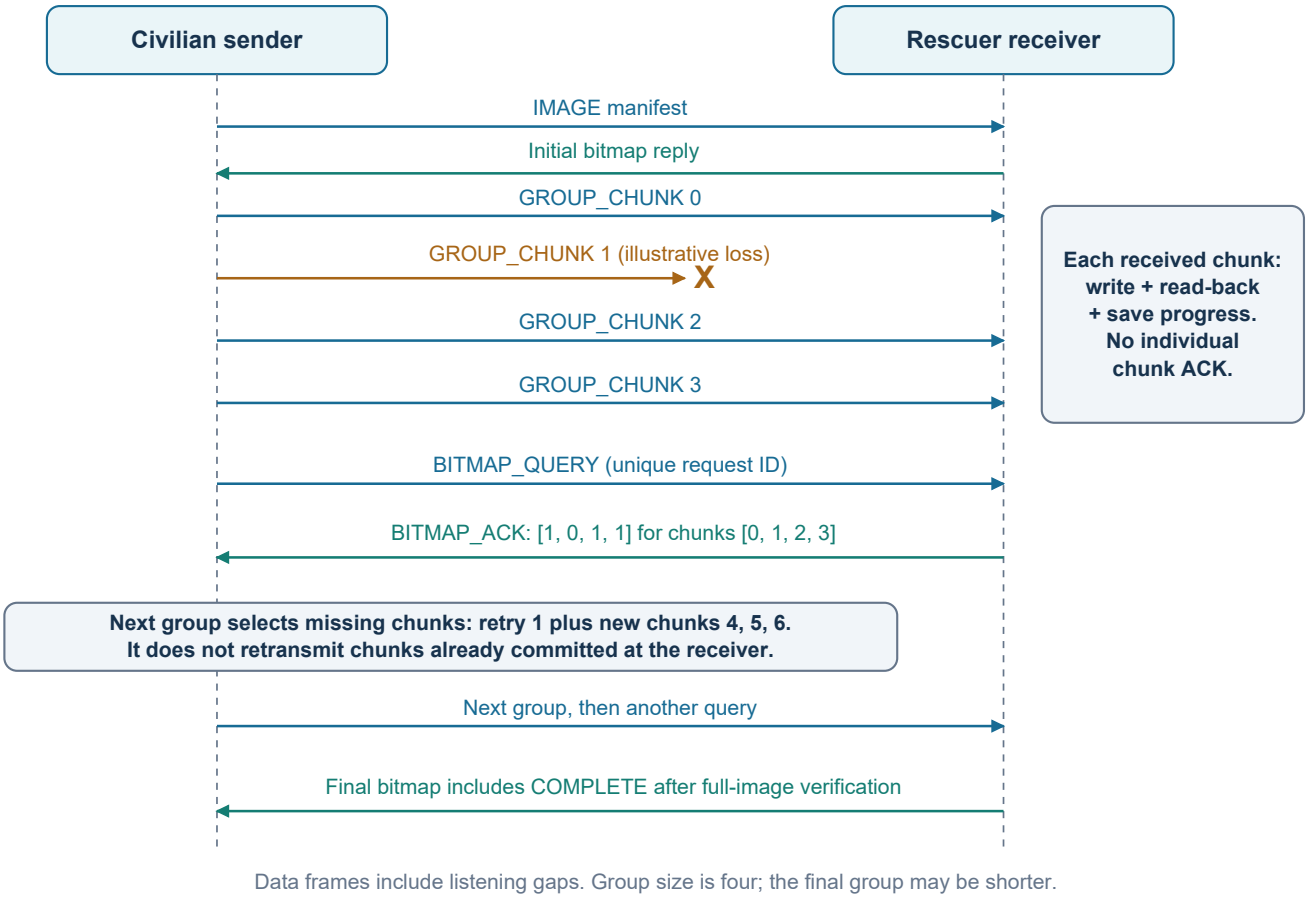



*Figure 7. Illustrative loss of chunk 1 in a four-chunk group. Bitmap positions shown in brackets follow the labelled chunk order, not a printed integer's bit order.*

After the initial manifest/bitmap exchange, the sender selects up to four currently missing chunks. GROUP_CHUNK frames are written and committed by the receiver without an individual ACK. The sender then issues BITMAP_QUERY; the receiver returns a full image bitmap and the exact query packet ID.

In the example, chunks 0, 2 and 3 are committed, while chunk 1 is absent. The next group can contain the missing chunk 1 together with new chunks 4, 5 and 6. Selective retransmission avoids re-sending chunks already confirmed. A matching reply can also reveal a receiver progress reset after failed CRC verification.

Grouped transfer is not continuous transmit-only operation. Each local data frame is followed by at least a 200 ms listening gap, with additional spacing for estimated relay hops. The four-chunk size stays fixed even when reception quality changes; the last group may contain fewer chunks.

If a reply is lost, the current group's bitmap is queried again. After four unanswered requests, the sender pauses and later recovers through a manifest exchange. Final delivery requires a full bitmap and COMPLETE, not merely successful transmission of the last radio frame. Grouping reduces ACK frequency but adds query frames and retains storage and listening costs.

Source trace: TransferConfig.h:15-27; LoRaMeshManager.h:187-244,351-403; StorageManager.h:224-253

# 9. Mode 4: adaptive policy

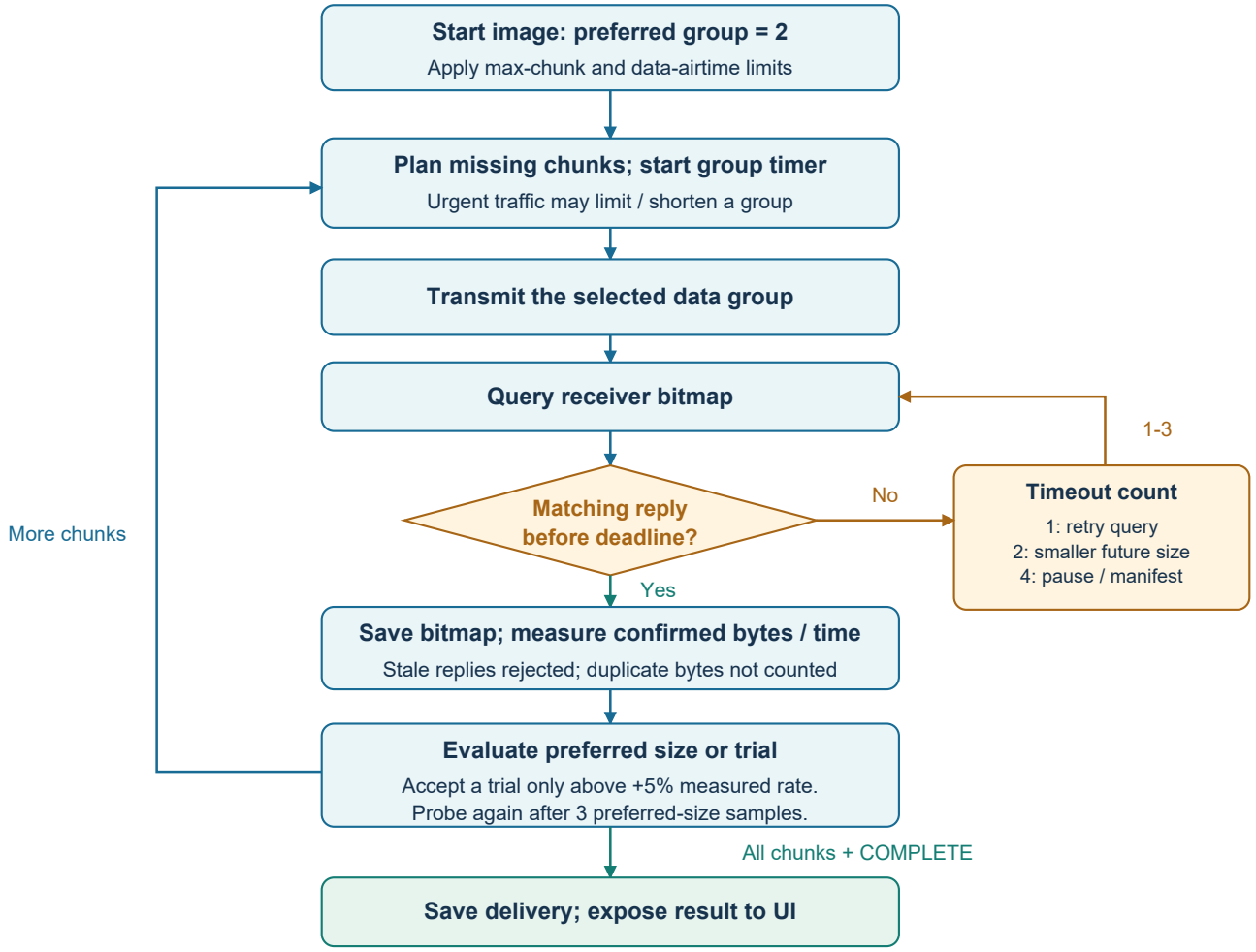



*Figure 8. Mode 4 uses the same grouped receiver as mode 3. Adaptation is a sender-side policy over group size, measured delivery rate and acknowledgement waiting.*

Each image starts with a preferred group size of two. A group's rate is newly confirmed JPEG bytes divided by elapsed time from planning until the sender saves the matching bitmap. Waiting for replies counts in that elapsed time. Already-confirmed bytes are excluded, and partial chunks contribute only their actual bytes.

After an initial sample, the policy tries a neighbouring size by doubling or halving it within bounds. It adopts a trial only when its measured rate exceeds the preferred estimate by more than 5%. Preferred-size samples update a weighted estimate (75% previous, 25% new), and another probe occurs after three such samples. Missing chunks prompt a smaller trial.

One unanswered request retries the query without shrinking the group; two permit a smaller preferred size for later groups. The already-sent group still needs bitmap recovery. Four unanswered requests enter pause/manifest recovery. The reply timeout tracks smoothed delay and variation, with a path/airtime floor and a 60 s ceiling.

Group size is capped by eight configured chunks and an 8 s data-airtime budget. At the current full-frame airtime this gives five chunks. Urgent traffic can limit the next group or close it at a frame boundary. Interrupted or short final groups do not train the policy. A valid full COMPLETE reply still ends the transfer even if its sample is ineligible for learning.

Source trace: TransferPolicy.h:38-155; TransferConfig.h:15-31; LoRaMeshManager.h:197-244,351-403

# 10. Persistent reception and ACK semantics

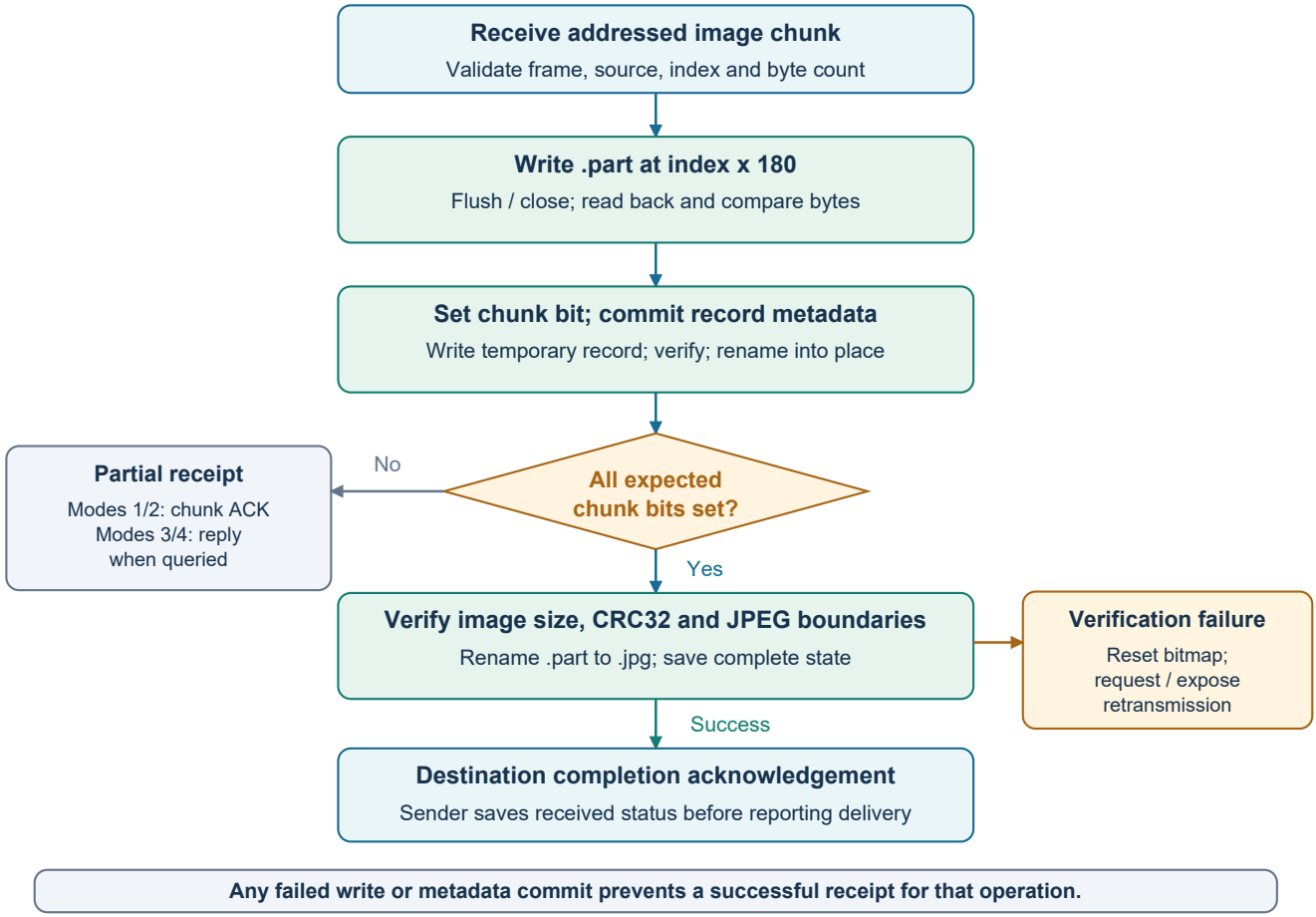



*Figure 9. Successful reception is tied to saved state. Full image verification and committed completion are required before final acknowledgement.*

A received frame is validated for packet structure, protocol version, destination and authentication before application processing. The image receiver checks the source, chunk index and expected byte count. The file offset is index x 180. A successfully written chunk is flushed, closed and read back before its bit is added to the record.

Record saving writes a temporary metadata file containing its own checksum, reads it back and replaces the destination record by rename. If this save fails, the operation does not receive a success acknowledgement. A duplicate committed chunk can be acknowledged without writing its image bytes again.

Once all expected chunks exist, the receiver checks byte length, CRC32 and JPEG start/end marker bytes. These checks establish expected file integrity and basic JPEG boundaries; they do not decode the image or prove perceptual usefulness. On a CRC failure, progress is reset and data is requested again. On success, the file is renamed to .jpg and completion metadata is saved.

The receiver can repair an interrupted finalisation where the completed JPEG exists but completion metadata is absent. Sender copies and receiver progress support retry/recovery, but finite storage, hardware faults and explicit Clear all operations remain relevant. Persistent acknowledgement is a defined delivery condition, not proof of zero data loss under every failure.

Source trace: StorageManager.h:154-171,180-253; LoRaMeshManager.h:154-244,326-344

# 11. Timing, evidence and research interpretation

**Separate the time intervals.** Phone capture/decode/compression, Wi-Fi upload, waiting for the SOS target, LoRa image transfer and browser refresh are distinct stages. A user-facing stopwatch can include all of them. Compare protocols with identical saved JPEG bytes and a consistent start/end definition; record compression and upload separately.

**Physical airtime remains.** A full current chunk frame contains a 39-byte application header, 3-byte chunk prefix, 180 JPEG bytes and an 8-byte authentication tag: 230 bytes supplied to the radio. At SF9, 125 kHz, coding rate 4/7 and an eight-symbol preamble, the code's airtime model gives approximately 1.574 s for that frame. For an 8192-byte image, data-frame airtime alone is approximately 71.8 s. This calculated example excludes manifests, queries, ACKs, listening gaps, retries and software delays.

**Why adaptive need not always win.** A short image may finish before enough groups are observed. A probe can cost time, and changing contention can make a previously faster size slower. Fixed groups may already suit a particular link. The 8 s cap applies to data airtime only, so it is not an upper bound on complete group time or urgent-message latency.

**Position the contribution accurately.** IEEE work already covers compression [1], a dual-receiver image architecture [2], multi-packet transmission [3] and bitmap-ACK image transfer [4]. The project-specific subject to evaluate is its measured-goodput group policy, query-first reply recovery and emergency-message scheduling on the present hardware. Architecture diagrams document the implementation; they do not prove novelty or superior performance.

**Current evidence.** The earlier 100 s and 74 s runs used different compressed images. No percentage improvement is established by that pair. Figures in this pack are explanatory schematics, and the lost packet in Figure 7 is an example rather than an observed test trace.

Source trace: PacketDefs.h:10-45; TransferPolicy.h:8-14; research/Methodology-IEEE-HD-draft.md

# 12. References and reproducibility

[1] Y. Onykiienko, S. Naida, M. Mazin, I. Kyrylenko, and P. Popovych, "An Efficient Image Compression Method for LoRa Networks," IEEE STEE, 2026, pp. TT2.14.1-TT2.14.5. DOI / IEEE publication

[2] S. De et al., "Interleaved Reception in LoRa Using IFT: A Hardware-Efficient Architecture for Fast and Low-Power Image Transmission," IEEE Internet of Things Journal, vol. 12, no. 19, pp. 39777-39787, 2025. DOI / IEEE publication

[3] J. Torres, M. Nunez, and J. M. Mauricio Villanueva, "LoRa Multi-Packet Protocol for Image Transmission," IEEE INSCIT, 2024, pp. 1-6. DOI / IEEE publication

[4] T. Chen, D. Eager, and D. Makaroff, "Efficient Image Transmission Using LoRa Technology In Agricultural Monitoring IoT Systems," IEEE iThings/GreenCom/CPSCom/SmartData, 2019, pp. 937-944. DOI / IEEE publication

All academic references are IEEE publications. References [1]-[3] were verified through publisher/institution abstracts and DOI metadata; the full author-hosted paper was reviewed for [4]. Only the verified broad scope of the recent papers is used. Detailed architecture descriptions come from this project's source, not from those papers.

The source manifest records SHA-256 hashes for the non-secret files inspected. It deliberately excludes certificate/private-key headers and NetworkSecrets.h. Source references identify the implementation inspected, not an assurance that a particular connected board is running that revision.

# Reuse in the paper

Number the selected figures consistently with the institutional template and introduce each in the text. Keep the definitions of saved locally, received by destination and human rescue response distinct. Include the rubric-required acknowledgement of AI assistance in diagram preparation, writing and code work, using the institution's required wording.